

Linear Data Structures, Trees & Binary Search Trees

Arrays

- Static array: contiguous memory, $O(1)$ random access via `base+i*size`; fixed capacity.
- Dynamic/elastic array: on overflow, allocate a NEW array of double capacity, copy all n elements across, free the old one $\rightarrow O(n)$ for that one resize, but it happens only once every $\sim n/2$ insertions.
- Amortized cost: n pushes with doubling cost $O(n)$ total copies ($1+2+4+\dots+n < 2n$) $\Rightarrow O(1)$ amortized per push (aggregate method).
- Shrink policy: halve capacity only once size drops to $1/4$ of capacity (NOT $1/2$) — prevents "thrashing" (grow/shrink repeatedly at the same boundary).
- Insert/delete at index i : shift every element after i by one slot — $O(n)$ worst case. Insert/delete at the END is $O(1)$ amortized (dynamic array only).
- Big-O: access $O(1)$ · search $O(n)$ unsorted / $O(\log n)$ sorted (binary search) · insert/delete-at-end $O(1)$ amortized · insert/delete-at-arbitrary-index $O(n)$.

Linked Lists

- Singly linked: `node = {data, next};` head pointer. Traversal $O(n)$. Insert-at-head $O(1)$. Insert-at-tail $O(n)$ unless a tail pointer is kept. Insert-AFTER a given node $O(1)$. Delete needs the PREVIOUS node — $O(n)$ to find it unless doubly linked.
- Doubly linked: `node = {data, prev, next}`. $O(1)$ delete given the node itself (no need to search for its predecessor). $O(1)$ insert-before/after a given node. Traversal in both directions.
- Circular linked list: `last.next = head` (and `head.prev = last` if doubly linked) — no NULL terminator, so traversal must explicitly stop on returning to the start node. Used for round-robin scheduling and circular buffers.
- Skip list: several "levels" of forward pointers stacked above a base sorted linked list. A new node's level is chosen randomly (coin-flip per level, $P(\text{level} \geq k) = 1/2^k$).
- Skip-list search: start at the top-left sentinel; move RIGHT while `next.key < target`; otherwise drop DOWN one level; repeat until level 0 — expected $O(\log n)$ search/insert/delete, purely probabilistic, no rotations at all.
- Array vs linked list: array $\rightarrow O(1)$ access, $O(n)$ middle insert/delete. Linked list $\rightarrow O(n)$ access, $O(1)$ insert/delete GIVEN the position/node. Choose by which operation dominates your workload.

WORKED MICRO-EXAMPLE — DOUBLING ARRAY

- Start capacity=1. Push 8 elements one by one: resize (copy+double) triggers exactly when full, at sizes 1,2,4,8 $\rightarrow$ copies made = $1+2+4+8 = 15$ for 8 pushes — under the $2 \times 8 = 16$ bound the aggregate method promises, so amortized cost/push < 2 , i.e. $O(1)$.

WORKED MICRO-EXAMPLE — LIST REVERSAL

- Iterative in-place reversal (3 pointers, $O(n)$ time, $O(1)$ space): `prev=NULL, cur=head; while cur≠NULL: { nxt=cur.next; cur.next=prev; prev=cur; cur=nxt }`; finally `head=prev`. Classic "every step" technique — losing the `nxt` pointer before overwriting `cur.next` is the #1 bug.

COMMON EXAM TRAPS

- Growth factor must be a CONSTANT MULTIPLE (e.g. $\times 2$), never " $+k$ " (linear growth) — linear growth gives $O(n)$ amortized, not $O(1)$.
- Circular-queue full/empty is an off-by-one classic — state clearly whether you use a count field or a wasted slot.
- Reversing a singly linked list needs 3 pointers (`prev`, `cur`, `nxt`), never 2 — otherwise the rest of the list is lost.
- Skip-list levels are chosen per NODE at insert time (independent coin-flips), not fixed globally.

Stacks (LIFO)

- `push / pop / peek / isEmpty` — all $O(1)$, array- or linked-list-backed.

APPLICATIONS

- Balanced parentheses: push every opening bracket; on a closing bracket, pop and check it matches; string is valid iff the stack is empty at the very end.
- Infix $\rightarrow$ Postfix (shunting-yard): scan left $\rightarrow$ right — operand $\rightarrow$ output; '(' $\rightarrow$ push; ')' $\rightarrow$ pop to output until '('; operator `o` $\rightarrow$ pop & output every stack-top operator with $\geq$ precedence (left-assoc, equal precedence included), THEN push `o`; at the end, pop all remaining operators to output.
- Postfix evaluation: scan left $\rightarrow$ right — operand $\rightarrow$ push; operator $\rightarrow$ pop 2 operands (b then a), compute `aopb`, push the result back.
- Function-call stack / recursion: each call pushes an activation record (locals + return address); return pops it — recursion depth is bounded by stack size (stack overflow on unbounded recursion).
- Iterative DFS: push the start node; loop: pop a node, process it, push its unvisited neighbours.
- Undo/redo history, syntax parsing, backtracking (N-Queens, maze solving) — anywhere "most recent first, then unwind" applies.

Queues (FIFO)

- enqueue at the rear, dequeue at the front — $O(1)$ with a linked list OR a CIRCULAR array.
- Circular array queue: `front/rear` indices wrap via `%capacity` — avoids the $O(n)$ shifting a plain array queue would need on dequeue. Distinguish full vs empty either by keeping an explicit count, or by always leaving one slot unused (`((rear+1)%cap==front  $\rightarrow$  full`).
- Deque (double-ended queue): $O(1)$ insert/delete at BOTH front and rear — generalises stack + queue in one structure.
- Priority Queue: dequeue returns the HIGHEST-PRIORITY element, not the oldest. Usually a array-based BINARY HEAP: `parent(i)= $\lfloor i/2 \rfloor$` , `children(i)= $2i, 2i+1$` .
- Heap insert = append at the end, then "sift UP" (repeatedly swap with parent while parent has lower priority). Heap extract-top = swap root with the last element, remove the last slot, then "sift DOWN" the new root (swap with the higher-priority child) — both $O(\log n)$.
- PQ applications: Dijkstra's shortest path & Prim's MST (always expand the cheapest frontier vertex), Huffman coding (repeatedly merge the 2 lowest-frequency nodes), OS process/CPU scheduling, top-k / k-nearest streaming queries, A^* search.

WORKED MICRO-EXAMPLE — INFIX $\rightarrow$ POSTFIX

- Infix: `A+B*C-D`. Scan: `A $\rightarrow$ out(A); $\rightarrow$ push(+); B $\rightarrow$ out(AB); $\rightarrow$ prec(*) $\rightarrow$ prec(+) so just push(*) [stack: + *]; C $\rightarrow$ out(ABC); $\rightarrow$ prec(-) $\leq$ prec(*) so pop* out(ABC*), prec(-) $\leq$ prec(+) so pop+ out(ABC*+), then push(-); D $\rightarrow$ out(ABC*+D). End: pop- out(ABC*+D-). Result: A B C * + D -.`

WORKED MICRO-EXAMPLE — POSTFIX EVAL

- Postfix `"6 2 3 + *"` $\rightarrow$ push 6; push 2; push 3; `"+" $\rightarrow$ pop 3,2 $\rightarrow$ push 5` [stack: 6,5]; `"*" $\rightarrow$ pop 5,6 $\rightarrow$ push 30`. Result = 30 (= $6 \times (2+3)$).

COMMON EXAM TRAPS

- Same-precedence, LEFT-associative operators (`+, -, *, /`) must still pop equal-precedence operators before pushing — only right-associative ops (e.g. exponent `^`) skip that pop.
- A priority queue is NOT a sorted array: insert is $O(\log n)$ via sift-up, not $O(n)$ via shifting.
- Stack-based DFS visits neighbours in the REVERSE of the order they were pushed (LIFO) — order can differ from recursive DFS unless you push carefully.
- Circular queue: BOTH front and rear must advance with `%capacity` — forgetting the modulo on either one is the most common bug.

Tree Terminology

- root (no parent) · leaf (no children) · internal node · parent/child/sibling · ancestor/descendant.
- `depth(node) = #edges from the root down to node`. `height(node) = #edges on the LONGEST path from node down to a leaf below it`. `height(tree) = height(root)`.
- `degree = #children of a node`; every node in a binary tree has `degree  $\leq 2$` .
- Full binary tree: every node has 0 or 2 children (never exactly 1).
- Complete binary tree: every level is full EXCEPT possibly the last, which is filled strictly LEFT-to-RIGHT with no gaps.
- Perfect binary tree: every internal node has 2 children AND every leaf sits at the same depth — exactly $2^{h+1}-1$ nodes for height h .
- Balanced / height-balanced: subtree heights differ by only a small bounded amount at every node (the exact bound depends on the structure, e.g. AVL requires ≤ 1).
- Degenerate / skewed tree: every node has ≤ 1 child — behaves exactly like a linked list, height = $n-1$.

Traversals (all $O(n)$)

- Preorder (Root, L, R): visit the node BEFORE its subtrees — used to COPY/serialise a tree, since the root is written before its children and the tree can be rebuilt by reading in this order.
- Inorder (L, Root, R): for a BST this visits keys in STRICTLY SORTED order — exactly the property used to validate that a tree is a correct BST.
- Postorder (L, R, Root): visit the node AFTER both subtrees — used to safely DELETE/free a tree bottom-up (children destroyed before their parent), or to evaluate an expression tree.
- Level-order (BFS): visit the root, then every depth-1 node, then depth-2, ... — implemented with a QUEUE: enqueue root; loop — dequeue a node, process it, enqueue its children.
- Recursive height(n): base case empty $\rightarrow -1$ (or 0, pick one convention and stay consistent); else $1+\max(\text{height}(n.\text{left}), \text{height}(n.\text{right}))$.
- Recursive size(n): 0 if empty, else $1+\text{size}(n.\text{left})+\text{size}(n.\text{right})$.
- Morris inorder traversal: threads temporary links (a node's inorder-predecessor's right pointer $\rightarrow$ back to the node) so the whole tree can be walked inorder in $O(1)$ EXTRA space (no stack, no recursion) — links are undone as they're used.

WORKED EXAMPLE — ONE TREE, 4 TRAVERSALS

- Tree: 4(2(1,3),6(5,7)). Preorder: 4,2,1,3,6,5,7. Inorder: 1,2,3,4,5,6,7 (sorted — it's a BST). Postorder: 1,3,2,5,7,6,4. Level-order: 4,2,6,1,3,5,7.

COMMON EXAM TRAPS

- Height and depth are measured from OPPOSITE ends of the tree (depth from the root down TO a node; height from a node down to its lowest leaf) — don't swap them.
- A tree with n nodes always has exactly $n-1$ edges, whatever its shape.
- "Balanced" has NO single universal definition — always say which rule you mean (AVL's $BF \in \{-1, 0, 1\}$? weight-balance? RB's equal black-height?).
- Level-order needs an explicit QUEUE; pre/in/postorder are naturally recursive (or need an explicit STACK if done iteratively) — mixing the two data structures up is a common slip.

RELATED STRUCTURES

- Threaded binary tree: NULL child pointers are reused ("threaded") to point to the node's inorder predecessor/successor instead — enables $O(1)$ -extra-space traversal without a stack/recursion (the same idea Morris traversal exploits temporarily).
- N-ary tree: each node may have MORE than 2 children (e.g. tries, B-trees). A binary tree is just the $n=2$ special case; traversal ideas (pre/post/level-order) generalise directly, but "inorder" is binary-tree-specific and has no single n-ary analogue.
- Binary Heap (array-backed complete binary tree, seen in Col 2) vs BST: a heap only orders parent-vs-child (weak, $O(1)$ find-min); a BST orders left-vs-node-vs-right everywhere (strong, $O(h)$ search) — different tradeoffs for different jobs.

Binary Search Trees

- BST property: for EVERY node, ALL keys in its left subtree $<$ `node.key` $<$ ALL keys in its right subtree — must hold at every node, not just immediate children (a common exam trap: check the WHOLE subtree, not just the direct child).
- Search(key): start at root; `key==node.key  $\rightarrow$  found`; `key<node.key  $\rightarrow$  go left`; `key>node.key  $\rightarrow$  go right`; hitting NULL $\rightarrow$ not found. Cost $O(h)$.
- Min = leftmost node (follow `.left` until NULL). Max = rightmost node (follow `.right` until NULL).
- Successor of x : if x has a right child $\rightarrow \text{min}(x.\text{right})$; else $\rightarrow$ the lowest ancestor of x whose LEFT child is also an ancestor of x (walk up while x is a right child). Predecessor is the mirror.

Insert

- Search for the key (guaranteed to fail); attach the new node as a LEAF exactly at the NULL pointer where the search fell off. Never restructures any existing node.

Delete — 3 cases

- Case 1, leaf (no children): unlink directly — set the parent's pointer to NULL.
- Case 2, one child: splice out — link the node's PARENT directly to its single CHILD, skipping the deleted node.
- Case 3, two children: copy the KEY of the in-order SUCCESSOR (= min of right subtree — predecessor = max of left subtree also valid) into this node, then recursively delete that successor node from the right subtree — guaranteed to land in case 1 or 2, since the minimum of a subtree never has a left child.

WORKED EXAMPLE — INSERT THEN DELETE

- Insert 50,30,70,20,40,60,80 in that order $\rightarrow$ tree: 50(30(20,40),70(60,80)). Delete(30) — two children — successor = `min(30.right)=40  $\rightarrow$  copy 40 into the node, delete the old leaf 40  $\rightarrow$  result: 50(40(20,—),70(60,80))`.
- Delete(50) (the ROOT, two children) uses the exact same rule: `successor = min(50.right) = 60  $\rightarrow$  root's key becomes 60, then delete the old 60 leaf`.

WHY BALANCE MATTERS

- All BST ops cost $O(h)$. Best case (balanced) $h=O(\log n)$. WORST case (degenerate — e.g. inserting already-sorted data) $h=O(n)$: the tree degrades into a linked list.
- This is exactly the motivation for AVL / Red-Black / Splay trees on page 2 — each adds a rebalancing rule so h stays $O(\log n)$ always (or amortized $O(\log n)$ for splay trees).

COMMON EXAM TRAPS

- The BST property must hold against the WHOLE subtree, not just the immediate child — a node can be individually "in order" with its parent yet still violate the property against a grandparent.
- Predecessor and successor are mirror images — mixing up which subtree (left vs right) you search is the most common slip.
- Deleting a node with two children NEVER creates a new violation, because the successor is copied by KEY, not by re-linking the whole subtree.

IMPLEMENTATION NOTES

- Iterative vs recursive: search/insert are naturally iterative (single downward loop, $O(1)$ extra space); delete's two-children case and any traversal are usually written recursively — an interviewer may ask you to convert either direction, so know both forms.
- Duplicate-key convention: a plain BST assumes distinct keys. If duplicates are allowed, PICK and STATE one rule consistently — e.g. "equal keys go right" — otherwise search/insert/delete become ambiguous about which of several equal nodes is meant.
- Augmented BSTs (mention only): store extra data per node — e.g. subtree SIZE ($\rightarrow O(h)$ order-statistics: find k -th smallest) or subtree HEIGHT ($\rightarrow$ feeds directly into AVL's balance factor on page 2).

AVL, Red-Black & Splay Trees

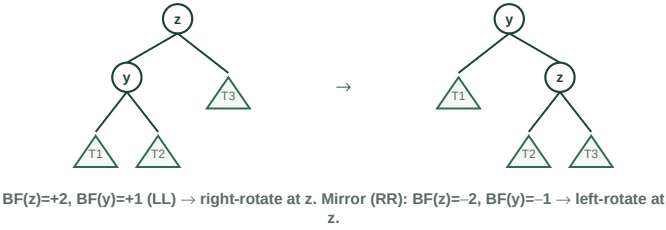
Comparison — BST vs AVL vs Red-Black vs Splay

Structure	Search	Insert	Delete	Height bound	Extra data/node	Best for
BST (plain)	$O(h)$	$O(h)$	$O(h)$	$O(n)$ worst (unbalanced / sorted-input case)	none	Simple lookups on mostly-random or static data; no rebalancing overhead.
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$\leq 1.44 \cdot \log_2(n+2)$	height or BF (small int)	Read-heavy: many lookups, few updates — strictest balance $\Rightarrow$ fastest search.
Red-Black	$O(\log n)$	$O(\log n)$	$O(\log n)$	$\leq 2 \cdot \log_2(n+1)$	1 colour bit	General-purpose / write-heavy — looser balance $\Rightarrow$ fewer rotations than AVL. Used in java.util.TreeMap, C++ std::map, Linux CFS scheduler.
Splay	$O(\log n)$ amortized	$O(\log n)$ amortized	$O(\log n)$ amortized	$O(n)$ worst single op	none — self-adjusts from access pattern	Locality-of-reference workloads / caches — recently/frequently used keys become $O(1)$ -ish.

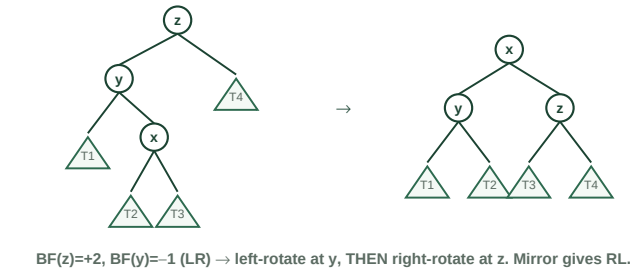
AVL Trees

- Balance factor $BF(x) = \text{height}(x.\text{left}) - \text{height}(x.\text{right})$. AVL invariant: $BF(x) \in \{-1, 0, 1\}$ at EVERY node, not just the root.
- Each node stores its own height (or BF directly) as extra data, updated bottom-up after any structural change.
- Height bound: a valid AVL tree of n nodes has height $\leq 1.44 \cdot \log_2(n+2) - 0.328 \approx O(\log n)$ — only ~44% taller than the perfect-tree minimum $\log_2(n+1)$.
- Insert: standard BST insert (always a new LEAF), then walk back UP to the root updating heights; the FIRST unbalanced node found (BF becomes ± 2) is fixed with exactly ONE single or double rotation — that always restores balance for the whole tree.
- Delete: standard BST delete (case 1/2/3, same as plain BST), then walk UP from the physically removed node's parent; UNLIKE insert, a fix at one level can un-balance the level above, so rotations may CASCADE all the way to the root.

ROTATION — LL / RR CASE (SINGLE)



ROTATION — LR / RL CASE (DOUBLE)



VINE METHOD (DAY–STOUT–WARREN)

- Rebalances an ENTIRE tree to near-perfect in $O(n)$, no per-node BF bookkeeping needed — good when many inserts happen before any query (batch-build / bulk-load).
- Phase 1 — vine: repeatedly right-rotate at the current node while it has a left child, else step right; collapses the tree into a sorted, fully right-leaning linked list (the "vine"), in $O(n)$.
- Phase 2 — compress: $m = 2^{\lceil \log_2(n+1) \rceil} - 1$ leftover nodes fixed first, then $m/2, m/4, \dots, 1$ — each pass left-rotates every other node along the vine, halving its length until it is a perfect (or near-perfect) tree.

COMMON EXAM TRAPS

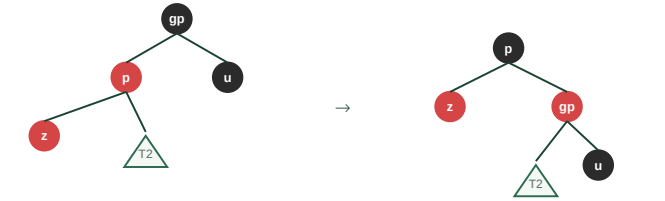
- A single rotation fixes AVL insert in $O(1)$ rotations total; delete can need $O(\log n)$ rotations (one per level) — do not assume delete is "just like insert."
- Diagnose LL vs LR (and RR vs RL) from the SIGN of the CHILD's balance factor, not just the unbalanced node's — same-sign $\Rightarrow$ single rotation, opposite-sign $\Rightarrow$ double.
- After ANY rotation, recompute heights of the rotated nodes bottom-up (child before parent) before continuing the walk upward.

Red-Black Trees

- 5 properties: (1) every node is RED or BLACK. (2) the root is BLACK. (3) every leaf (NIL/None) is treated as BLACK. (4) a RED node never has a RED child (no red-red edge, "no two reds in a row"). (5) every root-to-NIL path has the SAME number of BLACK nodes — this count (excluding the NIL itself) is the black-height, $bh(x)$.
- These 5 properties together guarantee height $\leq 2 \cdot \log_2(n+1)$ — looser than AVL's $BF \in \{-1, 0, 1\}$, which is exactly why RB needs fewer rotations per update.

INSERT-FIXUP — 3 CASES (Z = NEW RED NODE)

- Case 1, uncle RED: recolor parent & uncle BLACK, grandparent RED, then move z up to the grandparent and repeat — pushes the problem UP, no rotation yet.
- Case 2, uncle BLACK, z is an "inner" child (triangle shape): rotate at the PARENT to convert the triangle into a straight line — falls through into Case 3.
- Case 3, uncle BLACK, straight line: rotate at the GRANDPARENT, then swap colors of (old) parent and grandparent — done, loop terminates.



Case 3 (uncle u black, straight line): rotate at gp, recolor $p \leftrightarrow gp$. Case 2 first rotates at p to reach this shape.

DELETE-FIXUP — 4 CASES (X = DOUBLE-BLACK)

- Case 1, sibling RED: rotate at parent + recolor sibling/parent so the sibling becomes BLACK — converts into one of cases 2-4.
- Case 2, sibling BLACK with 2 BLACK children: recolor sibling RED, move the "double-black" defect up to the parent, repeat from there.
- Case 3, sibling BLACK, near child RED (far child BLACK): rotate at sibling + recolor — converts the shape into Case 4.
- Case 4, sibling BLACK, far child RED: rotate at parent + recolor sibling/parent/far-child to BLACK — defect resolved, done (terminal case).

RB VS AVL — QUICK FACTS

- RB allows up to $2\times$ the minimum height (looser) $\Rightarrow$ cheaper writes (≤ 2 rotations for insert-fixup, ≤ 3 for delete-fixup, though recoloring can cascade $O(\log n)$ levels with NO rotation cost).
- AVL is stricter/shorter $\Rightarrow$ faster reads. Rule of thumb: AVL for lookup-heavy, RB for update-heavy workloads.
- Every AVL tree is a valid (relaxed) RB-colorable tree, but not every RB tree satisfies AVL's BF bound.

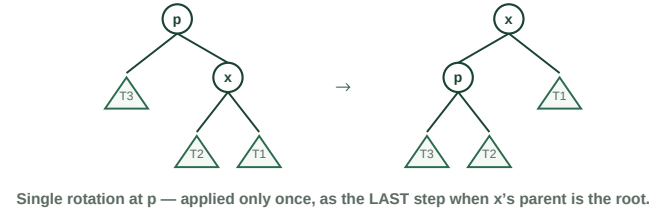
COMMON EXAM TRAPS

- NIL leaves count as BLACK and count toward black-height — forgetting them is the #1 source of miscounted black-heights.
- Recoloring alone (Case 1 insert, Case 2 delete) does NOT change the tree shape — only rotations do; don't draw a rotation where none happens.
- A red node's children can be BLACK internal nodes or BLACK NIL leaves — never assume a red node is childless.

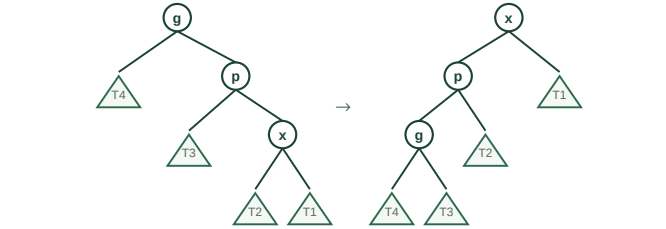
Splay Trees

- Self-adjusting BST: NO explicit balance field or invariant is stored. Every access (search, insert, or delete) ends by SPLAYING the touched node — repeated rotations that walk it all the way up to become the new ROOT.
- Splaying is done bottom-up, TWO levels at a time, choosing one of 3 cases based on the node x, its parent p, and grandparent g:

ZIG (X IS A DIRECT CHILD OF THE ROOT)

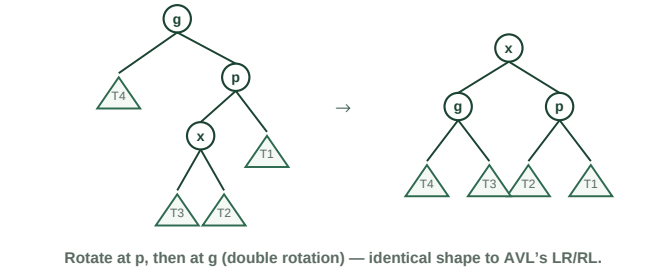


ZIG-ZIG (X AND P ARE BOTH LEFT, OR BOTH RIGHT CHILDREN)



Rotate at g FIRST, then at p (same direction twice) — NOT the same shape as zig-zag.

ZIG-ZAG (X AND P ARE ON OPPOSITE SIDES)



Rotate at p, then at g (double rotation) — identical shape to AVL's LR/RL.

- Access(key): ordinary BST search for key (or its would-be parent if absent), then splay whatever node the search ends on to the root.
- Insert(key): BST-insert the new leaf, then splay IT to the root (so the newest key is always instantly cheap to re-access).
- Delete(key): splay key to the root, detach it, leaving two subtrees L (all < key) and R (all > key); splay the MAXIMUM of L to L's root (it now has no right child), then attach R as its right child — a "join" via one extra splay.

AMORTIZED ANALYSIS — QUICK FACTS

- Potential function $\Phi = \sum$ over all nodes v of $\log_2(\text{size}(v))$ (size = #nodes in v's subtree) — pays for expensive splays using credit banked by earlier cheap ones.
- Access Lemma: the amortized cost of splaying x to the root is $\leq 3 \cdot (\log_2(\text{size}(\text{root})) - \log_2(\text{size}(x))) + 1 = O(\log n)$.
- Working Set Theorem: accessing an item costs $O(\log k)$ if only k DISTINCT items have been touched since its last access — splay trees adapt automatically to locality, with no configuration.
- No single access is guaranteed fast (one worst-case access can be $O(n)$ on a degenerate chain) — the $O(\log n)$ guarantee is only AMORTIZED over a sequence.

COMMON EXAM TRAPS

- Splaying happens on EVERY access, including a plain search that only reads a value — not just on insert/delete.
- Zig-zig $\neq$ zig-zag: zig-zig rotates at g then p (same side twice, straight-line collapse); zig-zag rotates at p then g (opposite sides, like AVL's double rotation). Drawing the wrong shape is the most common mistake.
- A single zig only ever happens once per splay operation, exactly when x's parent is already the root (odd-depth start).